cd /Users/krishnagupta/cryobalance mkdir -p docs cat << 'EOF' > docs/tutorial.html

Cryobalance User Guide

Orientation Anisotropy Diagnostic and Continuous Balancing Pipeline for Single-Particle Cryo-EM

Author: Krishna Kant Gupta • Version 0.1.0 • License: MIT

1. Overview

Cryobalance is a high-throughput Python utility designed to quantify viewing angle distributions and restore reciprocal-space sampling balance in single-particle cryo-electron microscopy (cryo-EM). Macromolecules adsorbed at the air–water interface often exhibit preferred orientation, leading to anisotropic Fourier sampling, resolution loss, and directional stretching in 3D density maps.

Cryobalance decomposes Euler viewing coordinates onto an orthonormal real spherical harmonic basis. It calculates rotationally invariant anisotropy metrics and assigns Tikhonov-regularized inverse-density weights to particles without discarding observations or altering alignment parameters.

2. Installation

Install the verified package directly through standard package managers:

```
pip install cryobalance
```

For development installation with test dependencies from source:

```
git clone https://github.com/krishnakantgupta-ai/Cryobalance.git
cd Cryobalance
pip install -e ".[dev]"
```

3. Command-Line Interface (CLI)

Cryobalance provides a standalone console command for rapid integration into automated preprocessing workflows:

```
cryobalance <input_particles.star> <output_balanced.star> [OPTIONS]
```

Command-Line Arguments & Flags

Option	Default	Description
`-l, --lmax`	`12`	Maximum spherical harmonic degree cutoff ($L_{\max}$). Controls angular resolution of the density fit.
`-s, --symmetry`	`C1`	Point-group symmetry of the macromolecule (e.g., `C1`, `C3`, `D2`, `D7`).
`-a, --alpha`	`0.02`	Tikhonov regularization damping factor (α). Prevents weight divergence in sparse regions.
`--help`	—	Display available options and usage summary.

Example Executions

```
# Basic run with C1 symmetry
cryobalance run_data.star balanced_data.star

# Symmetry-constrained run on a trimer (C3)
cryobalance run_data.star balanced_C3.star --symmetry C3 --lmax 12 --alpha 0.02
```

4. Quantitative Diagnostics Explained

Upon execution, Cryobalance reports three essential metrics for assessing dataset quality:

- **Anisotropy Index (A):** Ratio of anisotropic spherical harmonic power to isotropic monopole power. Values near `0.0` signify ideal isotropic sampling; values exceeding `0.65` indicate severe preferred orientation requiring mitigation.
- **Sampling Entropy ($\tilde{H}$):** Normalized spherical Shannon entropy measuring angular diversity over the unit sphere ($0.0 \le \tilde{H} \le 1.0$).
- **Effective Sample Size (ESS):** Evaluated via Kish's formula $\text{ESS} = (\sum w_i)^2 / \sum w_i^2$. Quantifies the statistical power of the reweighted ensemble after penalizing redundant views.

> **Note on Weight Regularization:** The default damping factor `alpha = 0.02` prevents noisy, isolated particles from receiving excessively large weights. For datasets with exceptionally few side views, `alpha = 0.04` offers more conservative regularized smoothing.

5. Downstream Reconstruction in RELION

Cryobalance writes balanced weights directly to the standard `rlnParticleWeight` column in the exported STAR file. RELION automatically incorporates this column during 3D back-projection without modifying camera calibrations or 2D alignments:

```
relion_reconstruct \
    --i balanced_data.star \
    --o balanced_reconstruction.mrc \
    --ctf \
    --sym c3 \
    --maxres 3.0
```

6. Python API Reference

You can also embed Cryobalance directly into Python scripts or analysis notebooks:

```
import numpy as np
import starfile
from cryobalance.core import compute_metrics_and_weights

# 1. Load STAR table
data = starfile.read("particles.star")
df = data["particles"] if isinstance(data, dict) else data

# 2. Extract angles in radians
phi = np.deg2rad(df["rlnAngleRot"].to_numpy()) % (2.0 * np.pi)
theta = np.clip(np.deg2rad(df["rlnAngleTilt"].to_numpy()), 0.0, np.pi)

# 3. Compute balancing weights
weights, metrics = compute_metrics_and_weights(
    phi=phi,
    theta=theta,
    l_max=12,
    sym="C3",
    alpha=0.02
)

# 4. Save updated metadata
df["rlnParticleWeight"] = np.round(weights, 5)
starfile.write(data, "balanced_particles.star", overwrite=True)
```

7. Verification & Tests

To verify package integrity and mathematical routines in your environment, run the automated test suite:

```
pytest tests/ -v
```

EOF echo "[✓] Saved docs/tutorial.html"